

Comandos para contenedores:

Crea y ejecuta un contenedor a partir de una imagen, con un nombre personalizado:

docker run --name <nombre_contenedor> <nombre_imagen>

Ejecutar un contenedor con y publicar un puerto(s) del contenedor al host.

docker run -p <puerto_host>:<puerto_contenedor> <nombre_imagen>

Ejecutar un contenedor en segundo plano

docker run -d <nombre_imagen>

Iniciar o detener un contenedor existente:

docker start|stop <nombre_del_contenedor> (o <id_del_contenedor>)

Eliminar un contenedor detenido:

docker rm <nombre_del_contenedor>

Abrir un shell dentro de un contenedor en ejecución:

docker exec -it <nombre_del_contenedor> sh

Obtener y seguir los logs de un contenedor:

docker logs -f <nombre_contenedor>

Inspeccionar un contenedor en ejecución:

docker inspect <nombre_del_contenedor> (o <id_del_contenedor>)

Para listar los contenedores actualmente en ejecución:

docker ps

Listar todos los contenedores docker (en ejecución y parados):

docker ps --all

Ver las estadísticas de uso de recursos

docker container stats

El comando es este:

docker run -it --rm -d -p 8080:80 --name web aminespinoza/sitioweb:1.0

Lo que hace, en resumen, es arrancar un contenedor Docker en segundo plano usando la imagen aminespinoza/sitioweb:1.0, le pone el nombre web y publica el sitio para que puedas verlo desde tu máquina en el puerto 8080.

Te lo desarmo pieza por pieza.

docker run

Le dice a Docker que cree y ejecute un contenedor nuevo a partir de una imagen.

-it

Aquí en realidad hay dos opciones juntas:

-i mantiene abierta la entrada estándar del contenedor.
t asigna una terminal interactiva.

Esto se usa mucho cuando quieres meterte al contenedor y trabajar “como si estuvieras dentro”. Pero aquí hay un detalle importante: este comando también lleva **-d**, así que **-it** queda medio raro. No está prohibido, pero sí es una mezcla poco útil en este caso. Es como ponerse traje y pijama al mismo tiempo.

--rm

Hace que Docker borre automáticamente el contenedor cuando se detenga.

Eso significa que si el contenedor se apaga o tú lo detienes, desaparece. Sirve para pruebas rápidas y para no dejar basura acumulada.

-d

Significa **detached**, o sea, ejecuta el contenedor en segundo plano.

Tu terminal queda libre y el contenedor sigue corriendo aparte.

-p 8080:80

Mapea puertos así:

8080 = puerto de tu máquina host

80 = puerto dentro del contenedor

Entonces, si dentro del contenedor hay un servidor web escuchando en el puerto 80, tú lo vas a abrir desde tu navegador entrando a:

http://localhost:8080

o si estás en otra máquina de la red:

http://IP_DEL_HOST:8080

--name web

Le pone al contenedor el nombre **web**.

Eso facilita administrarlo después, por ejemplo:

```
docker logs web
docker stop web
docker exec -it web sh
```

En vez de usar el ID largo del contenedor.

aminespinoza/sitioweb:1.0

Ese es el nombre de la imagen.

Se compone así:

`aminespinoza` = usuario o repositorio en Docker Hub

`sitioweb` = nombre de la imagen

`1.0` = etiqueta o versión

Si no la tienes descargada, Docker la baja primero y luego crea el contenedor con ella.

Entonces, dicho de forma corrida, el comando significa:

“Crea y ejecuta un contenedor llamado `web`, usando la imagen

`aminespinoza/sitioweb:1.0`, déjalo corriendo en segundo plano, expón su puerto web interno 80 a través del puerto 8080 de mi máquina, y bórralo automáticamente cuando se detenga.”

Ahora, lo más interesante: hay una contradicción práctica entre `-it` y `-d`.

`-d` dice: “corre en segundo plano”.

`-it` dice: “quiero sesión interactiva”.

Se puede usar junto, sí, pero normalmente no tiene mucho sentido para un servidor web. Para un contenedor web, lo normal sería algo más limpio como:

```
docker run --rm -d -p 8080:80 --name web aminespinoza/sitioweb:1.0
```

Y si quisieras entrar después al contenedor, entonces harías:

```
docker exec -it web sh
```

Eso sí tiene lógica.

También te conviene saber qué pasa cuando lo detienes:

```
docker stop web
```

Como usaste `--rm`, al detenerlo Docker lo elimina automáticamente. O sea, después de eso ya no aparecerá ni siquiera en `docker ps -a`.

Y para comprobar que quedó corriendo:

```
docker ps
```

Y para ver los logs:

```
docker logs web
```